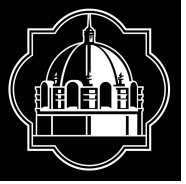
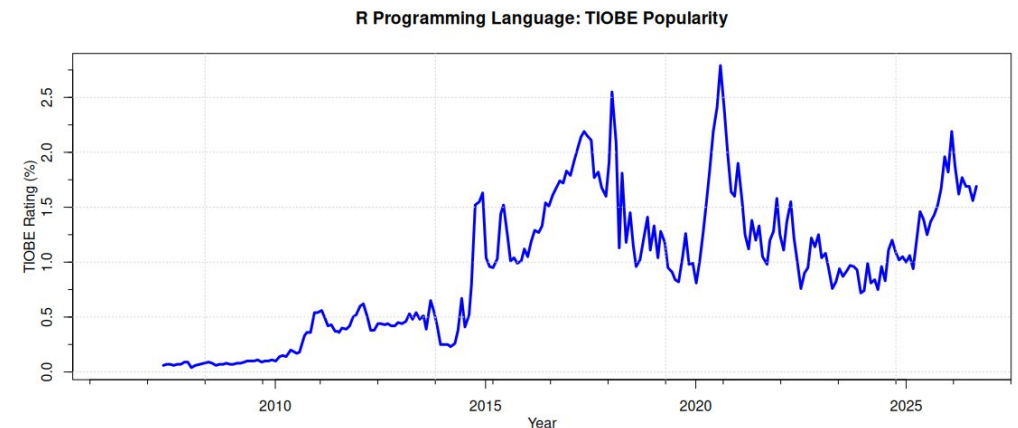
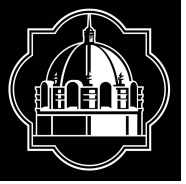


What is R



- R is a statistical and programming environment
 - Developed in 1993 by Ross Ihaka and Robert Gentleman
 - Inspired by the S programming language at AT&T Bell Labs (circa 1976)
 - Free and Open Source
- R is maintained by the R Core Team
 - Computer Scientists and Statisticians
 - Inner Code team that maintains full control of the core software
- R Consortium
 - Funding is obtained from corporate membership
 - R Posit, Microsoft, Glaxo Smith Kline (GSK), Novartis, ESRI,...
 - Support the tooling and infrastructure of R
- CRAN – Comprehensive R Archive Network
 - Mirrored repositories that allow you to download libraries





Downloading and Running R

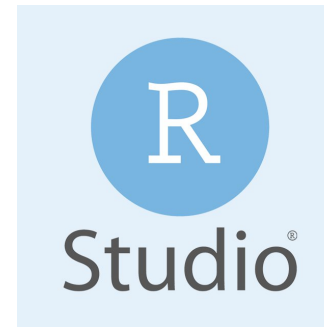
- R is a Free and Open Source Software
 - <https://cran.r-project.org/>
 - An updated version is released every 6 months
- R base can run from the command line and a text-editor
 - Computationally most efficient
 - Not easy though
- RStudio is a popular IDE to run R
 - IDE —> Interactive Development Environment
 - Need to have R installed first
- R can be run on desktops, HPC and the Cloud



Download and Install R

Precompiled binary distributions of the base system and contributed packages,

- [Download R for Linux](#) ([Debian](#), [Fedora/Redhat](#), [Ubuntu](#))
- [Download R for macOS](#)
- [Download R for Windows](#)

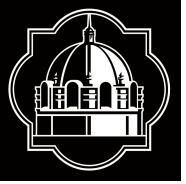


posit.co/downloads



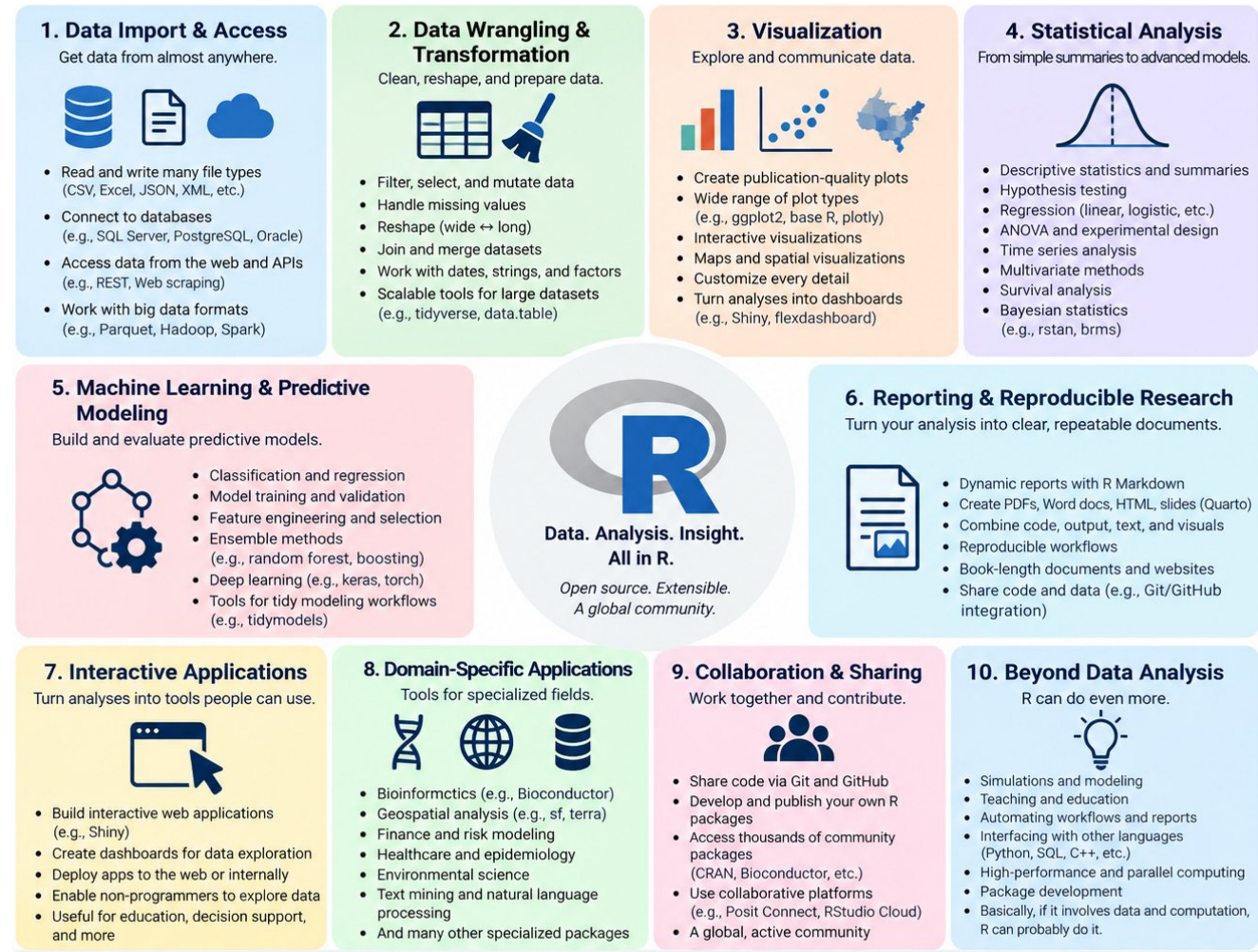
- Type posit.cloud as web address
- Sign up and Sign In (can use 3rd party authentication like Google, Github for signing in)

R Ecosystem

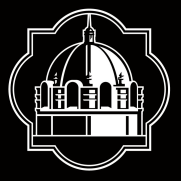


• R Ecosystem

- **Base R** – Basic Input Output, Visualization and Statistical testing and coding
 - Required
- **Libraries** – Extend the functionality of R
 - Some included with Base R but need to download as needed
 - Over 8000 external libraries
- **Repositories** – Download libraries and datasets
 - CRAN, Bioconductor are two main repositories to download libraries
 - Github also has several libraries
 - You can create your own personal library or contribute it to CRAN
- **Interfaces** – Ways to interact with R
 - R CLI, RGUI, RStudio and other help you work with R
- **Communication** – Tools that allow communication of the outputs and results
 - Quarto, Markdown, LaTeX – That allow R scripts to be embedded



R Computational Philosophy

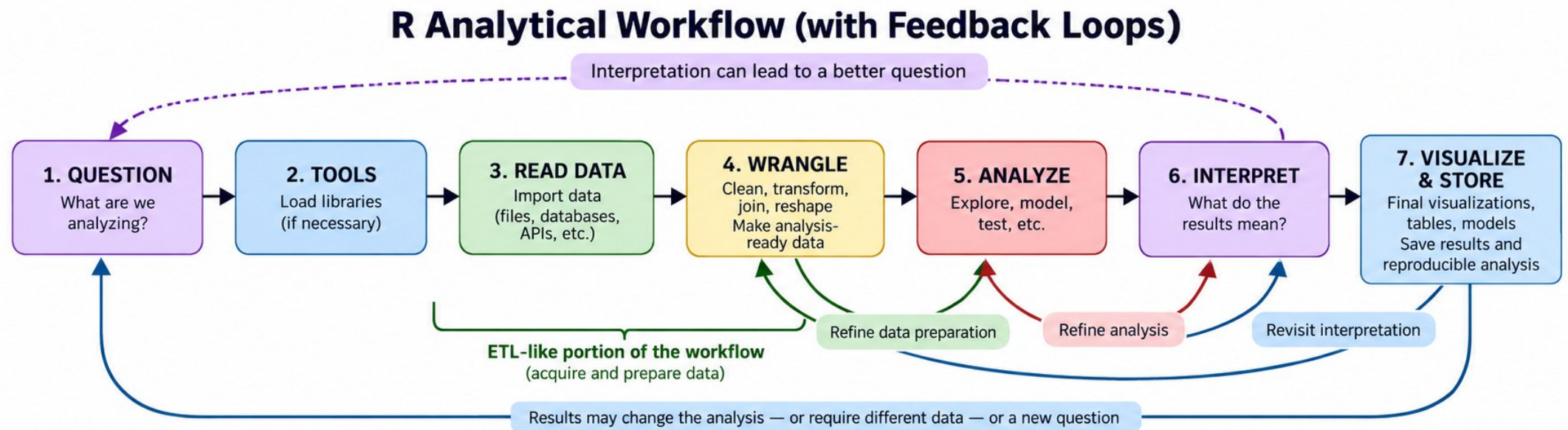
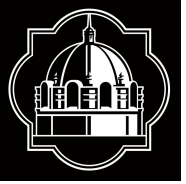


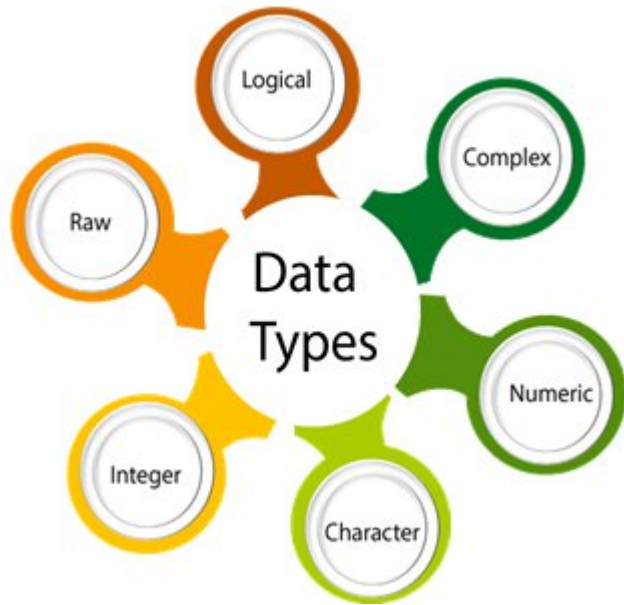
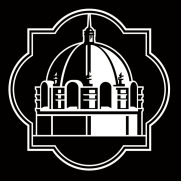
- R is object based

- Everything in R is an object
 - Different objects have different set of properties and characteristics
- R uses functional programming extensively
 - Functions perform a set of well defined tasks
- R is vector-oriented
 - Functions can be applied to a collection of similar objects all at once
- R has a programming language
 - You can write functions, libraries and other scripts to automate your analysis
 - Some R libraries and functionalities are written using R programming language
- R emphasizes interactivity
 - Analyze —> Visualize —> Interpret —> Refine
- R emphasizes reproducibility
 - Two analysts using the same data and conducting same analysis should get the same results
 - Scripts, functions, notebooks can be shared

R is interpretive
language - So you
can run the code
line by line

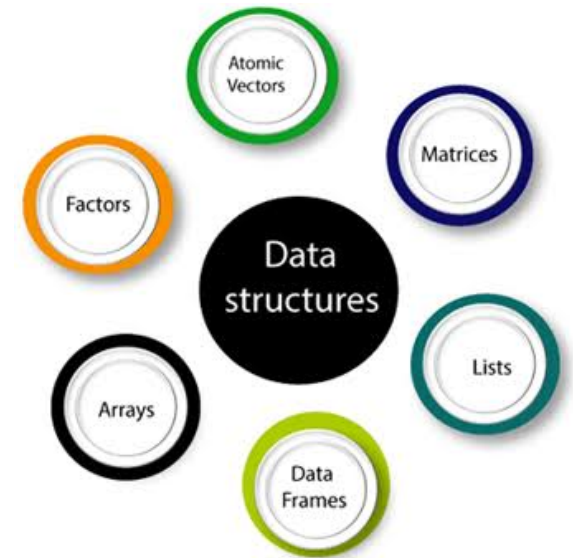
R Workflow

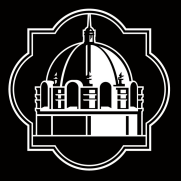




R Data Types and Structures

How does R organize data?





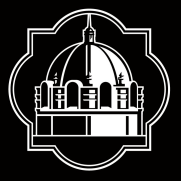
Data Types and Data Structures

- Data Type – What type of Data it is?
 - How much memory is needed?
- Data Structure – How are the data organized?
 - Useful for multidimensional data
 - Structure helps manage memory better

Data type — What is the data?	Example
Numeric	3.14
Integer	5L
Character	"Texas"
Logical	TRUE
Complex	2 + 3i

Data structure How is it organized?	Example
Vector	c(1, 4, 7)
Matrix	rows × columns of same type
Array	multidimensional collection
Data frame	columns of potentially different types
List	collection of almost anything

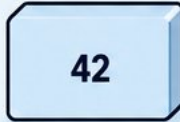
Data Types are ingredients and Data Structures are containers



Constant

A single, atomic value.

- Stores one value.
- Has a single type.
- No internal structure.
- Think of it as a single piece of data.



Value: 42
Type: numeric
Class: numeric

R examples

```
x <- 42  
y <- "Hello"  
z <- TRUE
```

Common R types for constants

numeric, character, logical

Homogeneous Data Structure

A collection of values, all of the same type.

- Stores multiple values.
- All elements have the same type.
- Can be one-dimensional (vector) or multi-dimensional (matrix, array).
- Think of it as a container of similar items.

Vector (all numeric)



Type: double
Class: numeric
Length: 5

R examples

```
v <- c(1, 2, 3, 4, 5)      # numeric vector  
s <- c("a", "b", "c")     # character vector  
m <- matrix(1:6, nrow = 2) # numeric matrix
```

Common R types (homogeneous)

vector, matrix, array

Heterogeneous Data Structure

A collection of values that can have different types.

- Stores multiple values.
- Elements can have different types.
- More flexible structure.
- Think of it as a container of different kinds of items.

List (mixed types)



Type: list
Class: list
Length: 4

R examples

```
lst <- list(42, "R", TRUE, 3.14)  
df <- data.frame(  
  id = 1:3,  
  name = c("Ann", "Bob", "Cat"),  
  score = c(88.5, 92, 76)  
)  
# data frame (columns can have different types)
```

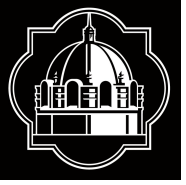
Common R types (heterogeneous)

list, data frame

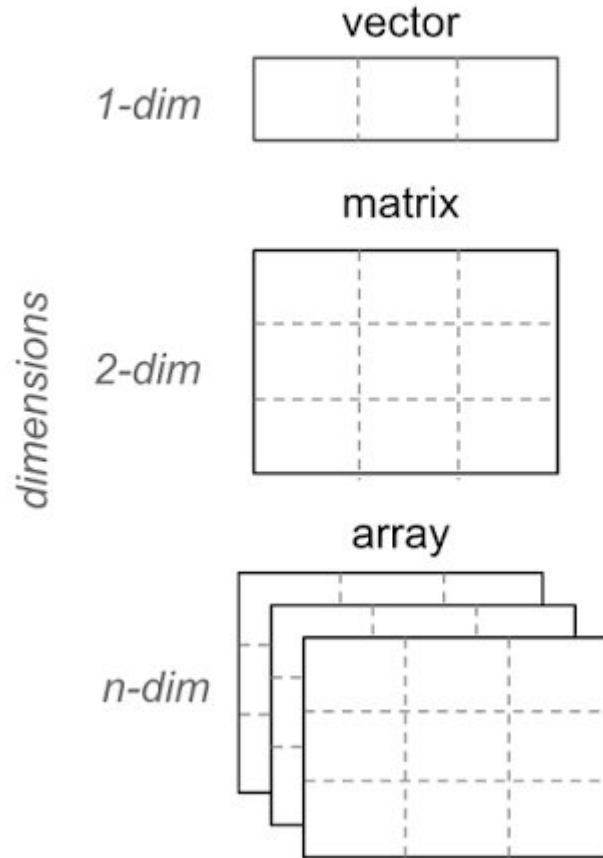
Summary

A constant stores one value. A homogeneous data structure stores multiple values of the same type.
A heterogeneous data structure stores multiple values that can be of different types.

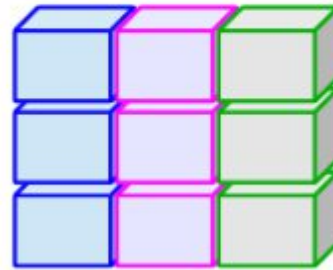
Multidimensional Data



Homogeneous Data Type

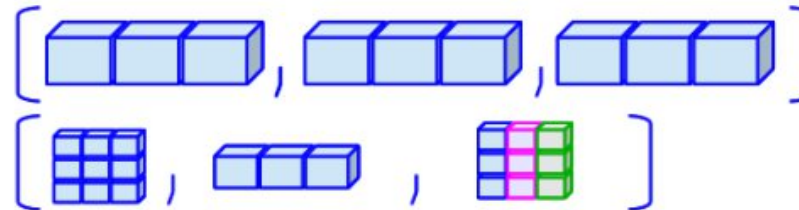


Data Frame
(Table)



Store Data in Rows x Columns
Each Column has the same data type

Lists

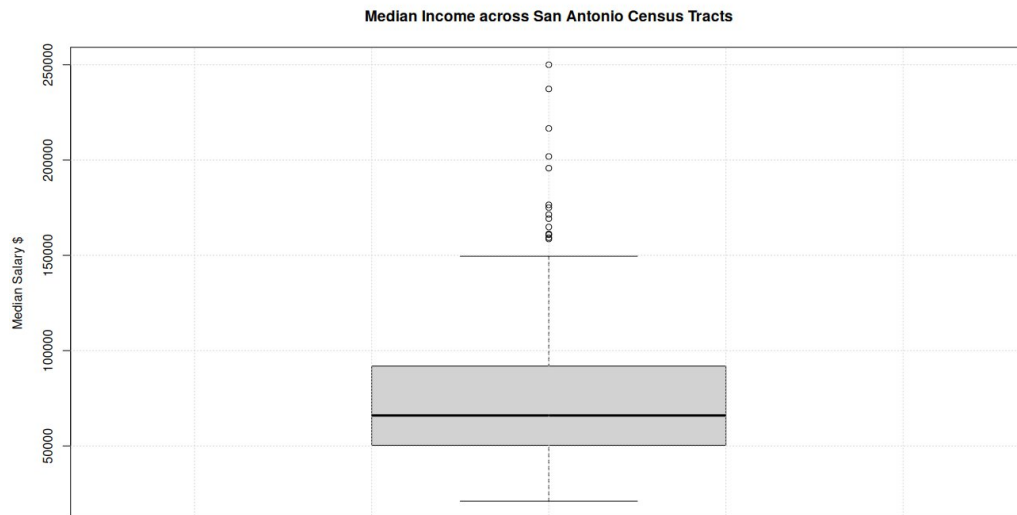


Flexible 1D
Container can
contain data of
any type or
dimension as its
elements



Reading and Analyzing Tabular Data

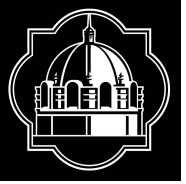
- Tabular data are most common
 - The best way to store it is as a csv file
 - Binary formats like parquet, feather, DuckDB for big data



```
1 # Read sanantonioacs.csv data and make a plot
2
3 #set working directory
4 path <- '/home/vuddameri/MathWorkshop/Data-Lecture1/Lecture1-Data/'
5 setwd(path)
6
7 # Read the Data
8 fname <- 'SanAntonioACS.csv'
9 a <- read.csv(fname)
10
11 # show some sample
12 head(a)
13
14 # Get column names
15 colnames(a)
16
17 # Make a Boxplot of Median Income
18 boxplot(a$MedIncome,ylab='Median Salary $',xlab="")
19 title('Median Income across San Antonio Census Tracts',cex.head=0.8)
20 grid()
```

	A	B	C	D	E	F
1	Tract	Population	MedAge	MedIncome	PerCapIncome	PovertyRate
2	Tract:1807.01	4128	33.3	44422	35842	24
3	Tract:1718.02	5725	36.9	44865	19608	27.9
4	Tract:1603	5712	37	61743	23246	27.7
5	Tract:1816.02	5830	33.3	56875	28032	11.9
6	Tract:1605.01	4772	17.6	20984	11387	72.4
7	Tract:1817.18	5012	42	95144	38594	5.5
8	Tract:1708	1742	25.9	35227	15679	28
9	Tract:1615.03	5746	32.4	59464	23442	18.2
10	Tract:1602	2769	40.3	43547	26049	14.3
11	Tract:1218.12	6347	40.1	100668	37973	7.3

Data Structures for Images

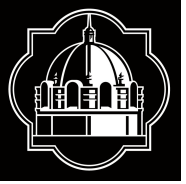


- R can read and manipulate image data
 - Remote sensing data
 - Aerial photographs
 - Ground-based photographs
- It can read data in multiple file formats
 - GIF, JPEG, PNG, BMP, TIFF
- Image data are essentially
 - Matrix for gray scale images
 - Array for color images
 - Header information stores meta data

```
# Example 2: Read Tiff Images
library(tiff)
fname1 <- 'NAIP102016TAMUSA.tif'
naip2016 <- readTIFF(fname1)
#Look at its characteristics
class(naip2016) # what data structure
typeof(naip2016) #what data type
dim(naip2016) # how many bands
plot(as.raster(naip2016[, , 1:3])) # make a plot using RGB
```

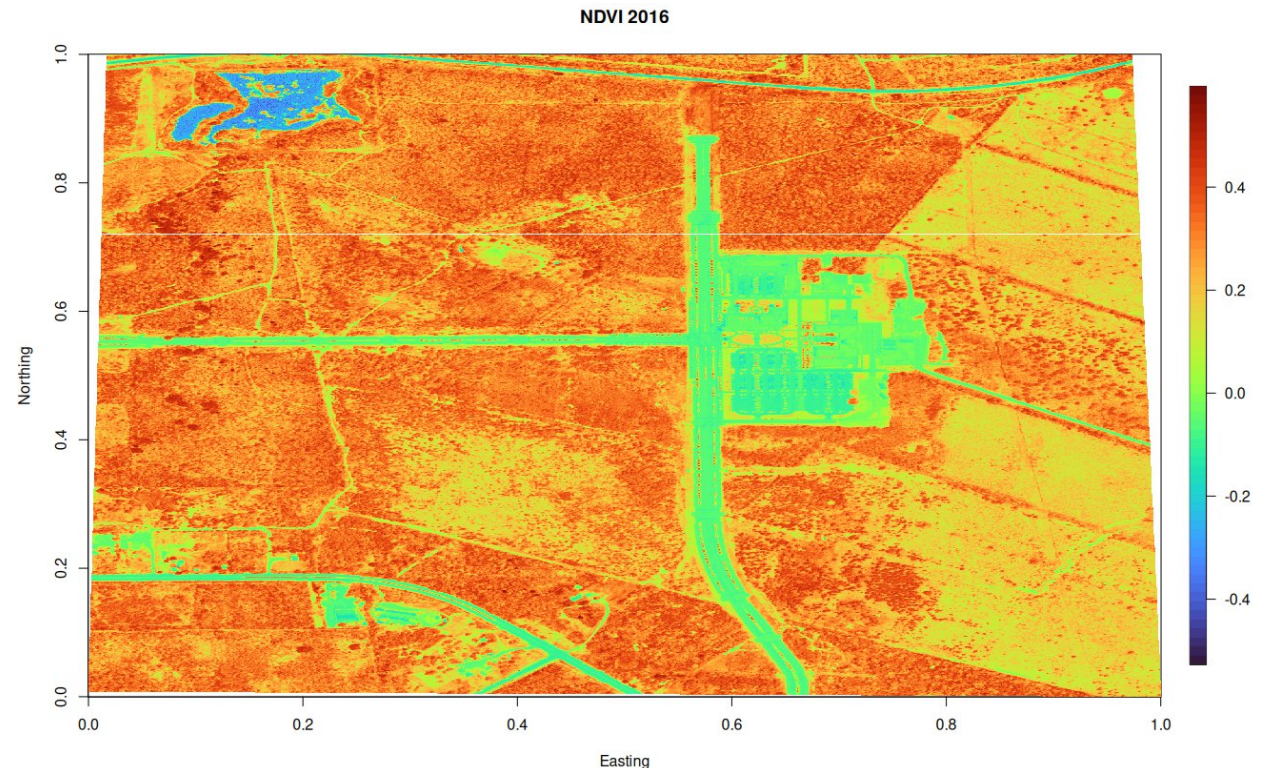


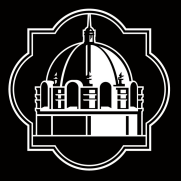
Image Manipulation



- Images are arrays that have same rows and columns
 - Pixels
- We can manipulate them pixel by pixel to obtain derived values
 - R functions are vectorized so we don't need to write loops
 - Vectorized function —> Same function applied to each element

```
# Example 3: Perform NDVI calculation
# Plot NIR
nir2016 <- naip2016[, , 4] # NIR Band
red2016 <- naip2016[, , 1] # Red Band
ndvi2016 <- (nir2016 - red2016) / (nir2016 + red2016)
image.plot(ndvi2016, main = "NDVI 2016",
           xlab = "Easting", ylab = "Northing")
```

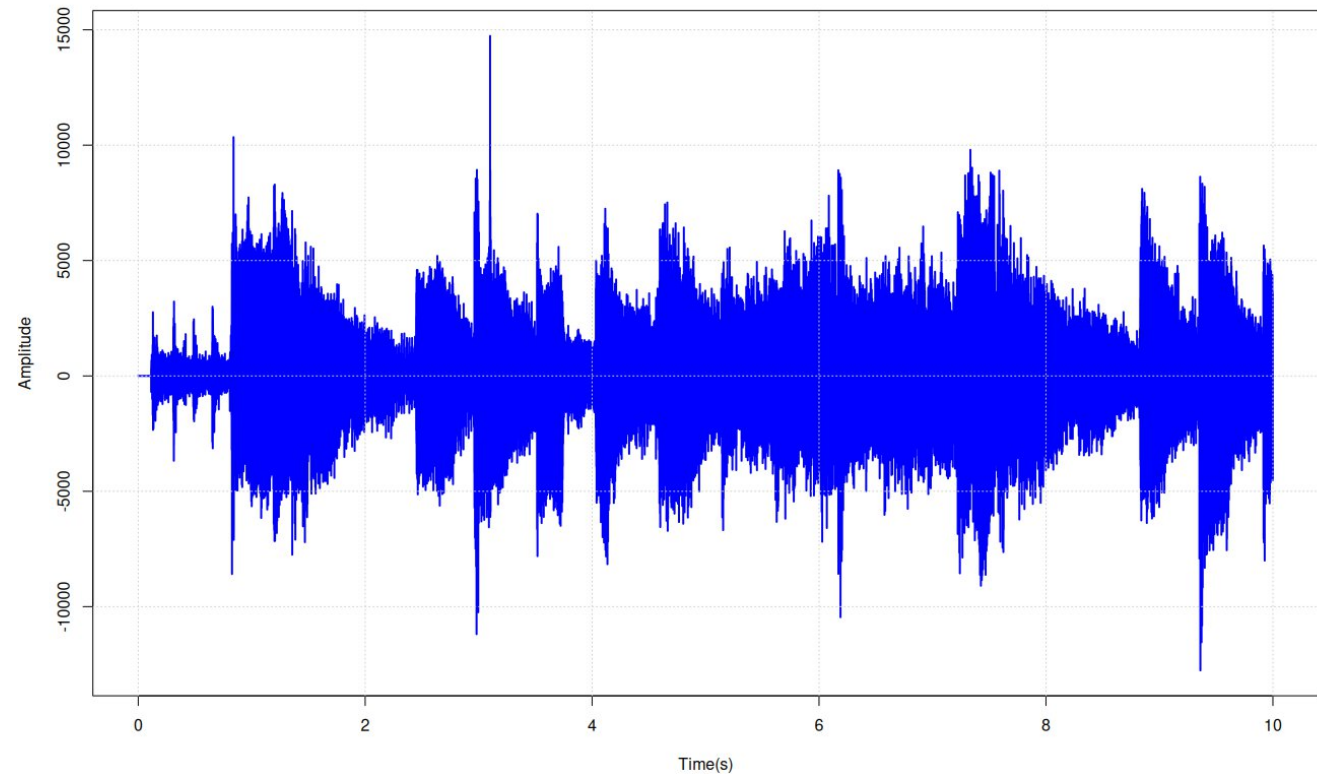




Audio Files

- R can read compressed audio files and extract numerical information
 - Audio amplitude
 - Sampling time
- Audio amplitude can be treated as any other numerical data and manipulated
 - Digital mixing (remixing), filtering, thresholding

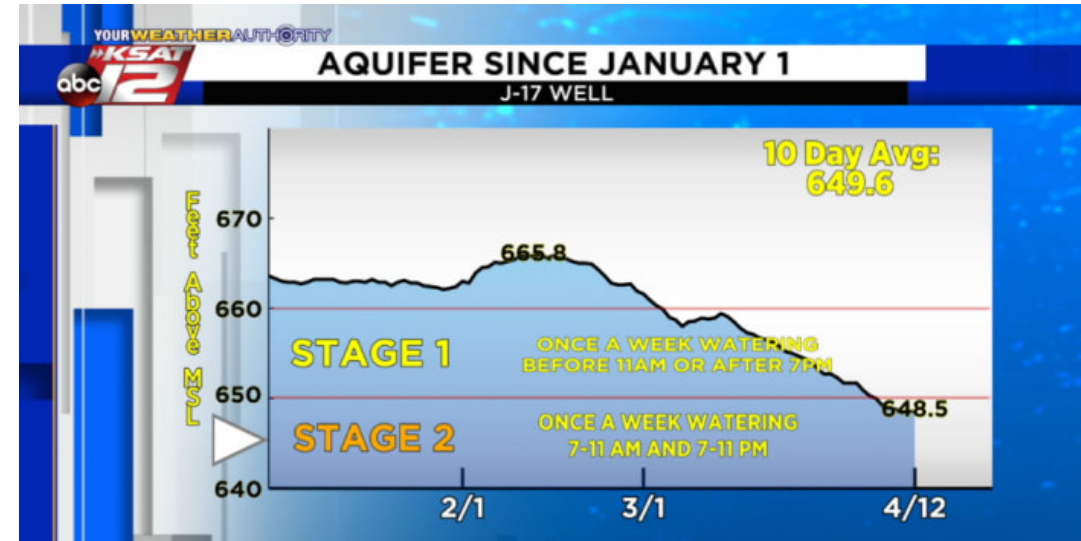
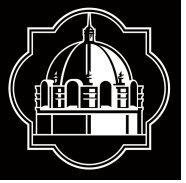
```
# Example 4: Read a MP3 Sound File
library(tuneR)
fname2 <- 'alamostrong.mp3'
audio <- readMP3(fname2) # read the file
x <- audio@left # get amplitudes
fs <- audio@samp.rate # get sampling rate
x10 <- x[1:(10*fs)] #keep first 10 sec
time <- seq(0,10*fs-1,1)/fs #get time
plot(time,x10,xlab='Time(s)',ylab='Amplitude',
      type='l',lwd=2,col='blue') # make plot
grid() # add grid
```



Time-Series Data Structures

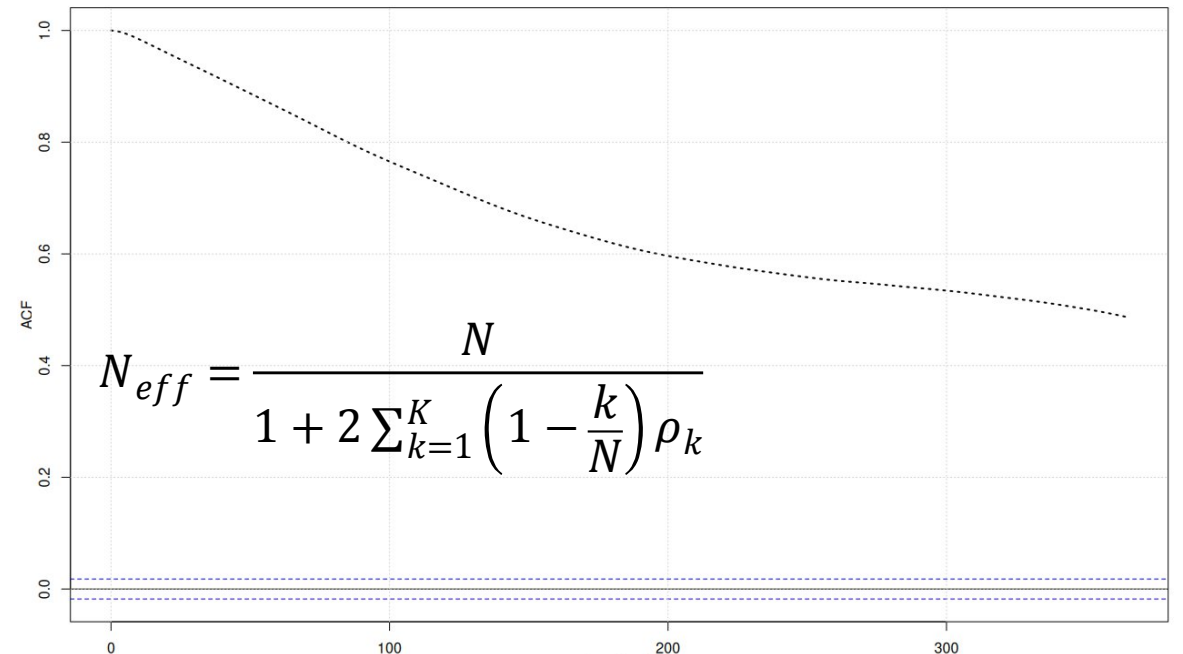
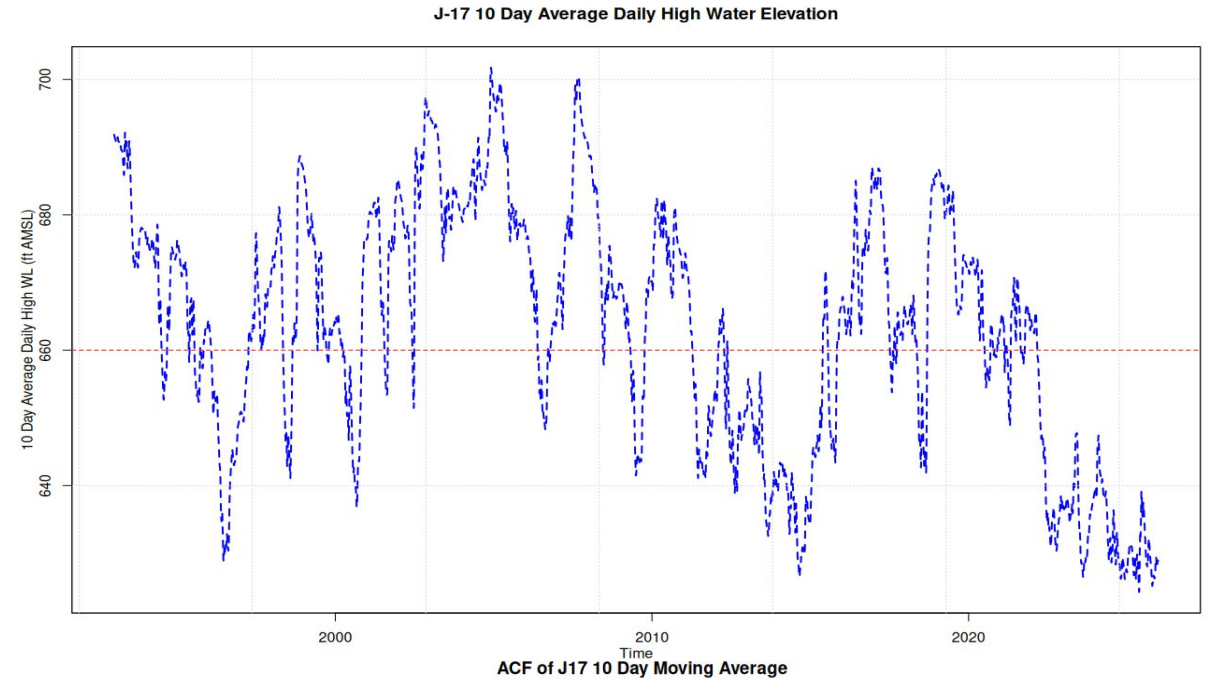
- Data Collected over time
 - Data typically from sensors
 - Missing values are common
- Resolution of the data depends upon the frequency of collection
 - Data collected at shorter intervals —> Higher resolution
 - Frequency is the reciprocal of time
- Time series data are useful to assess long-term trends, seasonal effects
 - Traffic over weekend vs on working days
- Time series data are highly autocorrelated
 - Current value is strongly correlated to previous values
- Sample size has considerable redundancy
 - Effective sample size <<< total sample size

Base R: ts, zoo
and xts packages

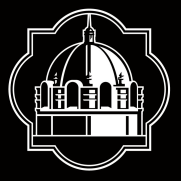


Time Series Persistence

```
#Example 5: Time Series Data
library(zoo)
library(forecast)
fname3 <- 'J17Well.csv'
wl <- read.csv(fname3)
head(wl)
WL <- wl$DHE # extract Daily high WL
date <- seq(as.Date('1993-01-01'),as.Date('2025-12-31'),freq=1) #create date sequence
WLTS <- zoo(WL,order.by=date) # create ts object
# Create a 10 day moving average
wlma10 <- rollmean(WLTS, k = 10, fill = NA)
# plot 10-D Moving average time-series|
par(mgp=c(2,1,0))
plot(wlma10,xlab='Time',
     ylab='10 Day Average Daily High WL (ft AMSL)',
     col='blue',lty=2,lwd=2,
     main = 'J-17 10 Day Average Daily High Water Elevation',cex.title=0.8)
grid()
abline(h=660,lty=2,lwd=1,col='red')
# Compute ACF
wlma10acf <- Acf(wlma10,lag.max=365)
plot(wlma10acf,type='l',xlab='Lag (days)',ylab='ACF',lty=3,lwd=2,main="")
grid()
title('ACF of J17 10 Day Moving Average',cex.title=0.8)
```

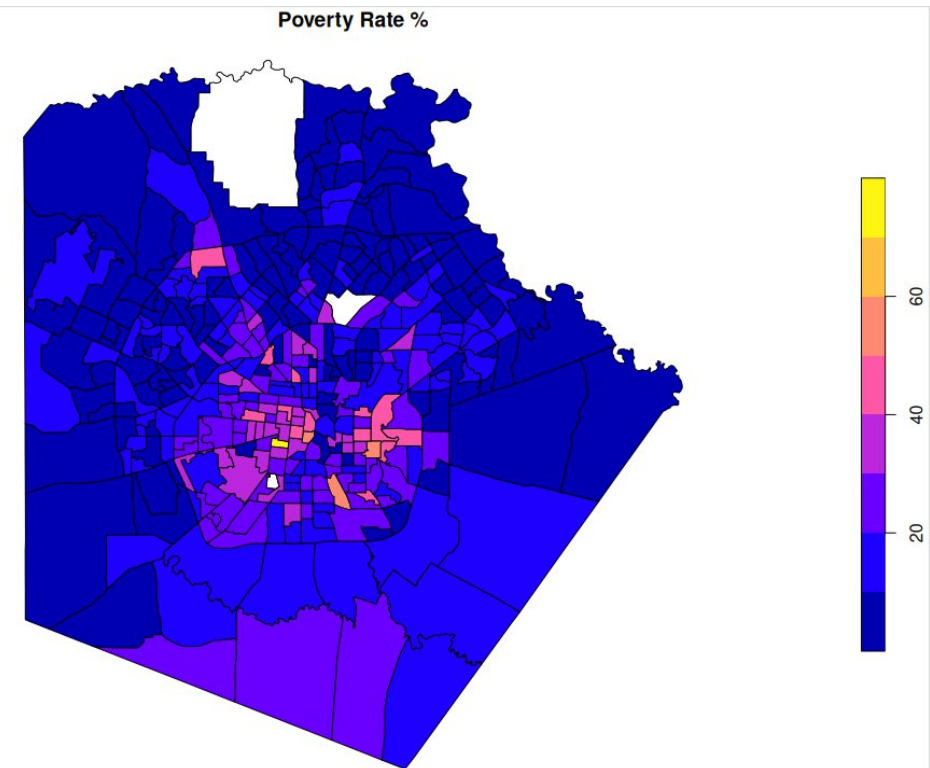


Spatial Data

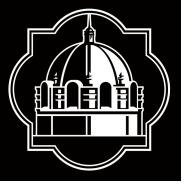


- R support GIS style analysis
 - Two main libraries - 'sf' and 'terra'
 - Depend internally on other software (GDAL)
- Spatial Data Structures
 - Data
 - Coordinate Reference System (CRS)
 - Geometry information – type, pixel size, boundings
- GIS has developed several file formats
 - .shp, .gpkg, GeoTiff, etc.
 - R can read and manipulate them
- R write the data as data.frame
 - Exposes GIS data to other analysis tools
 - Regression, Machine learning

```
# Example 6: GIS in R
library(sf)
fname4 <- 'SanAntonioACS.gpkg'
sa <- st_read(fname4)
head(sa)
plot(sa["PovertyRate"],main = "Poverty Rate %")
```



Spatial Data - Rasters



```
# Example 7: Vegetation changes around TAMUSA
library(terra) # Load library terra for raster
# Read NAIP images
naip16 <- rast("NAIP102016TAMUSA.tif")
naip24 <- rast("NAIP102024TAMUSA.tif")
compareGeom(naip16, naip24)
# resample to make them both extents
naip24a <- resample(naip24, naip16, method = "bilinear")
compareGeom(naip16, naip24a) # check crs and extent

# NAIP: Band 1 = Red, Band 4 = NIR
ndvi16 <- (naip16[[4]] - naip16[[1]]) /
  (naip16[[4]] + naip16[[1]])
ndvi24 <- (naip24a[[4]] - naip24a[[1]]) /
  (naip24a[[4]] + naip24a[[1]])

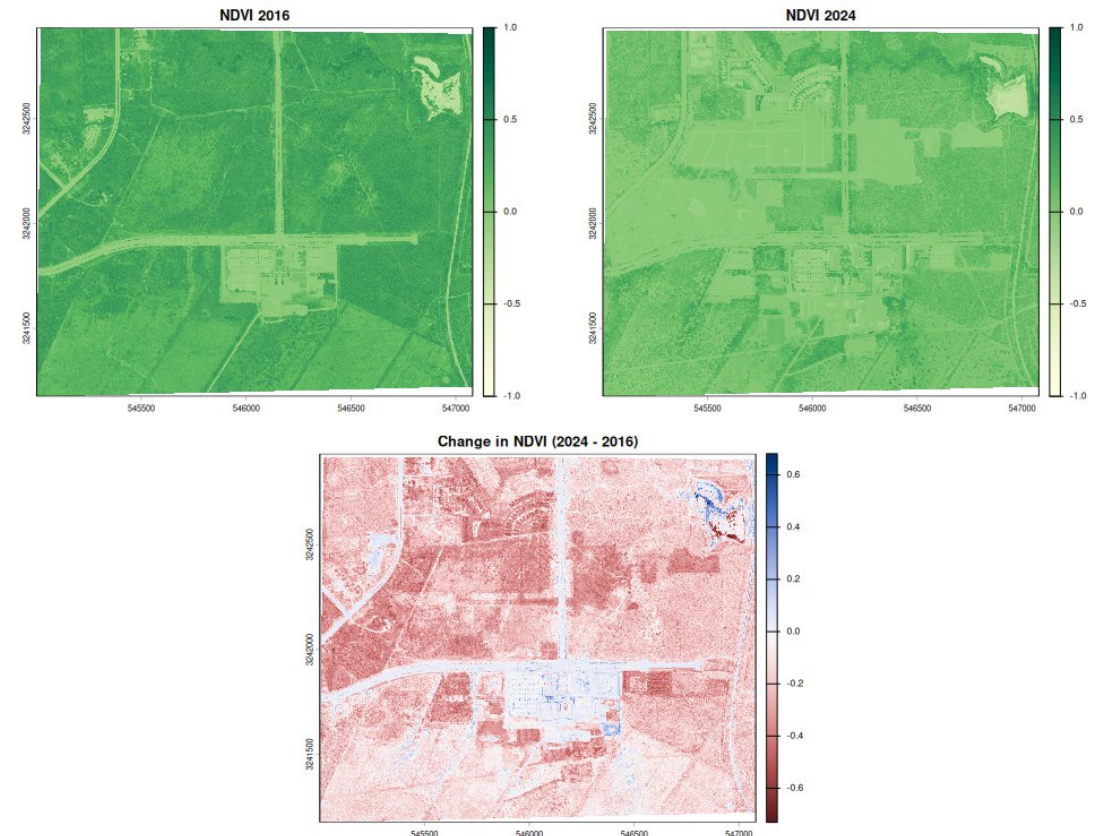
# Change: 2024 minus 2016
ndviChange <- ndvi24 - ndvi16

# Make plots
ndviCols <- rev(hcl.colors(100, "YlGn"))
layout(matrix(c(1, 1, 2, 2, 0, 3, 3, 0), nrow = 2,
  byrow = TRUE))
plot(ndvi16, col = ndviCols, range = c(-1, 1), main = "NDVI 2016")

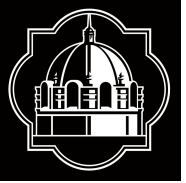
plot(ndvi24, col = ndviCols, range = c(-1, 1), main = "NDVI 2024")

plot(ndviChange, col = hcl.colors(100, "Blue-Red 3", rev = TRUE),
  main = "Change in NDVI (2024 - 2016)")
```

Urbanization around TAMUSA



Unstructured Data



- Unstructured data is more prevalent than structured data
 - News articles
 - Report and documents
 - Social Media Posts
- Unstructured data is typically in text format
 - Data structures build around character vector
- Data can be read in R
 - Data has to be structured for analysis

Text Mining converts textual information to numerical values to perform analysis

Text organization	R structure	Example/use
Single piece of text	Character scalar	One sentence/document
Collection of text	Character vector	Sentences, tweets, reviews
Text + attributes	Data frame	Tweet + date + author + sentiment
Hierarchical text	List	Documents containing paragraphs/sentences
Word counts	Table / matrix	Document-term frequencies
NLP corpus	Corpus object	Collections of documents
Document-term representation	DTM/TDM	Rows = documents, columns = terms
Modern ML representation	Numeric matrix	Embeddings

Text Mining in R

- R has several libraries
 - syuzhet is a popular library
 - Sentiment analysis



Newsletters Sign in

Hail, floods threaten Texas' fall forecast as severe storm odds spike

The next few months could be dicey, y'all.

By **Zachary-Taylor Wright**, Trending Digital Reporter
Sep 11, 2026

[MY SA Flood Story](#)



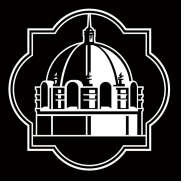


Hail, floods threaten Texas' fall forecast as severe storm odds spike. The next few months could be dicey, y'all.

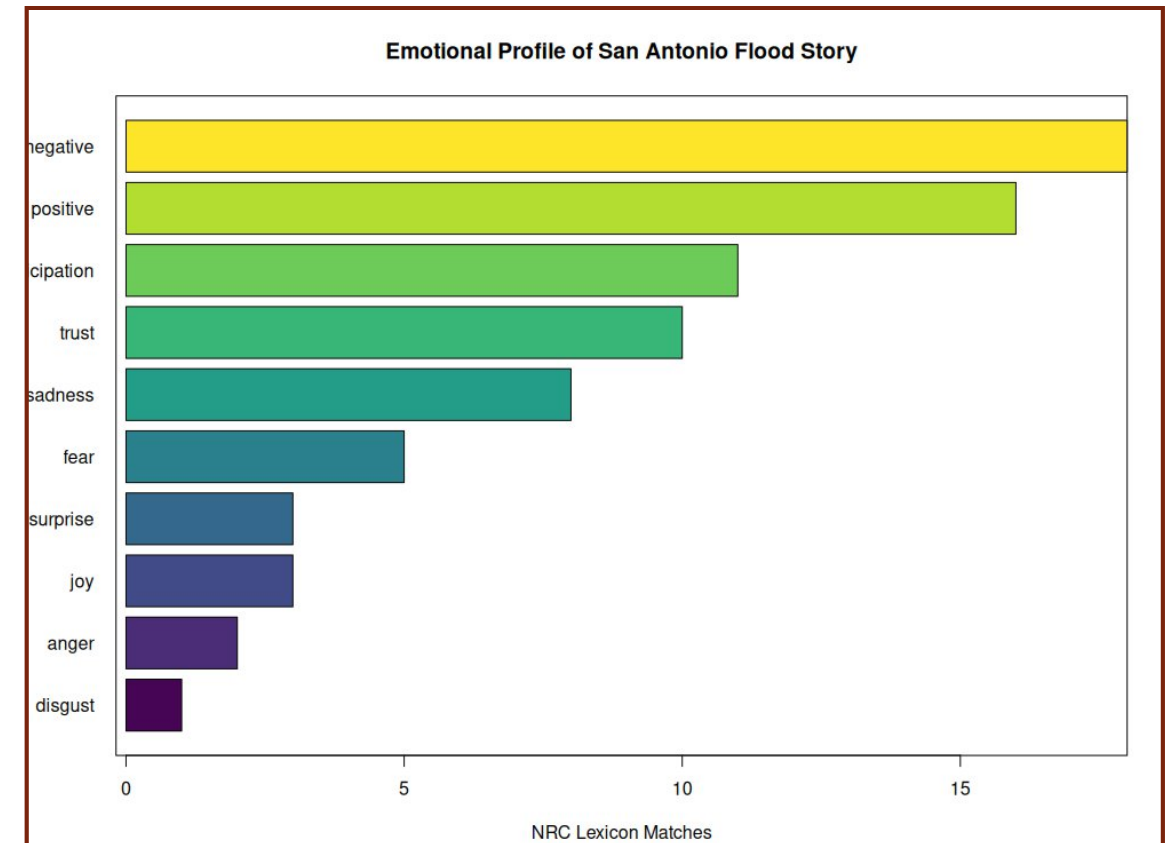
While it may not feel like it now, Texas could be in store for a major cooldown in the coming months. But with "Super El Niño" in full swing, the National Weather Service warns that those chillier days and cold fronts could come with hail, flooding and severe storms. During a fall outlook update, Mack Morris, a meteorologist with the National Weather Service Austin-San Antonio office, painted a harrowing picture for the coming months. Apparently, a strong El Niño can mean a fall season with double the average rainfall in Texas. Though it's surprisingly difficult to track a trend, as there have only been nine strong El Niño events since the 1950s, according to Morris.

"Typically, El Niño is more active in terms of overall severe reports," Morris said. "Not usually as much in terms of tornadoes, but hail and wind are higher in terms of the reports compared to La Niña." Fall-season hail reports nearly triple when comparing El Niño and La Niña years. However, these strong warm Pacific waters typically bring only a couple more hail reports than a neutral year. Severe weather reports are likely to spike this fall. If the stars align, Texas could see a fall and winter season that's as much as 6 degrees cooler than average. But what does all this rain mean for the persisting Texas drought? Some regions of the state, such as the Texas Hill Country and Edwards Plateau, have experienced flooding rains that lifted them out of persistent drought conditions. The Canyon Lake reservoir is "completely filled up" after the heavy dousing over the summer, says Morris. But huge chunks of the state are still struggling with clear skies and sunny days. An active fall and winter season - lingering until spring of 2027 - may finally spell an end to the parched earth in the Lone Star State. But that's yet to be solidified.

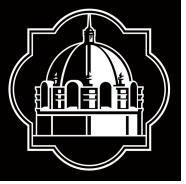
Text Mining R



```
# Example 8: Sentiment analysis
library(syuzhet)
fname5 <- "SAwinterfloods.txt"
txt <- readLines(fname5, warn = FALSE, encoding = "UTF-8")
head(txt)
corpus <- paste(txt, collapse = " ") # collapse into a single corpus
sentences <- get_sentences(corpus) # break into sentences
nrc <- get_nrc_sentiment(sentences) # perform sentiment analysis
totals <- colSums(nrc) # overall emotions totals
totals.sort <- sort(totals) # sort emotions
barplot(totals.sort, horiz = TRUE, las = 1,
        col = viridis(length(totals.sort)),
        xlab = "NRC Lexicon Matches",
        main = "Emotional Profile of San Antonio Flood Story")
box()
```



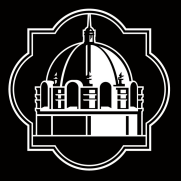
Genomics Data



- R offers a rich data structures for genomics data
- Able to read standard genomic data formats
 - FASTA, FASTQ

Genomic data	Typical structure	R/Bioconductor structure	Major libraries
DNA/RNA sequences	Character-like sequences	DNASTring, RNASTring, DNASTringSet	Biostrings
Genomic locations	Chromosome + start/end	GRanges	GenomicRanges
Sequence reads	FASTA/FASTQ records	ShortReadQ, DNASTringSet	ShortRead, Biostrings
Gene-expression data	Matrix + metadata	SummarizedExperiment	SummarizedExperiment, DESeq2

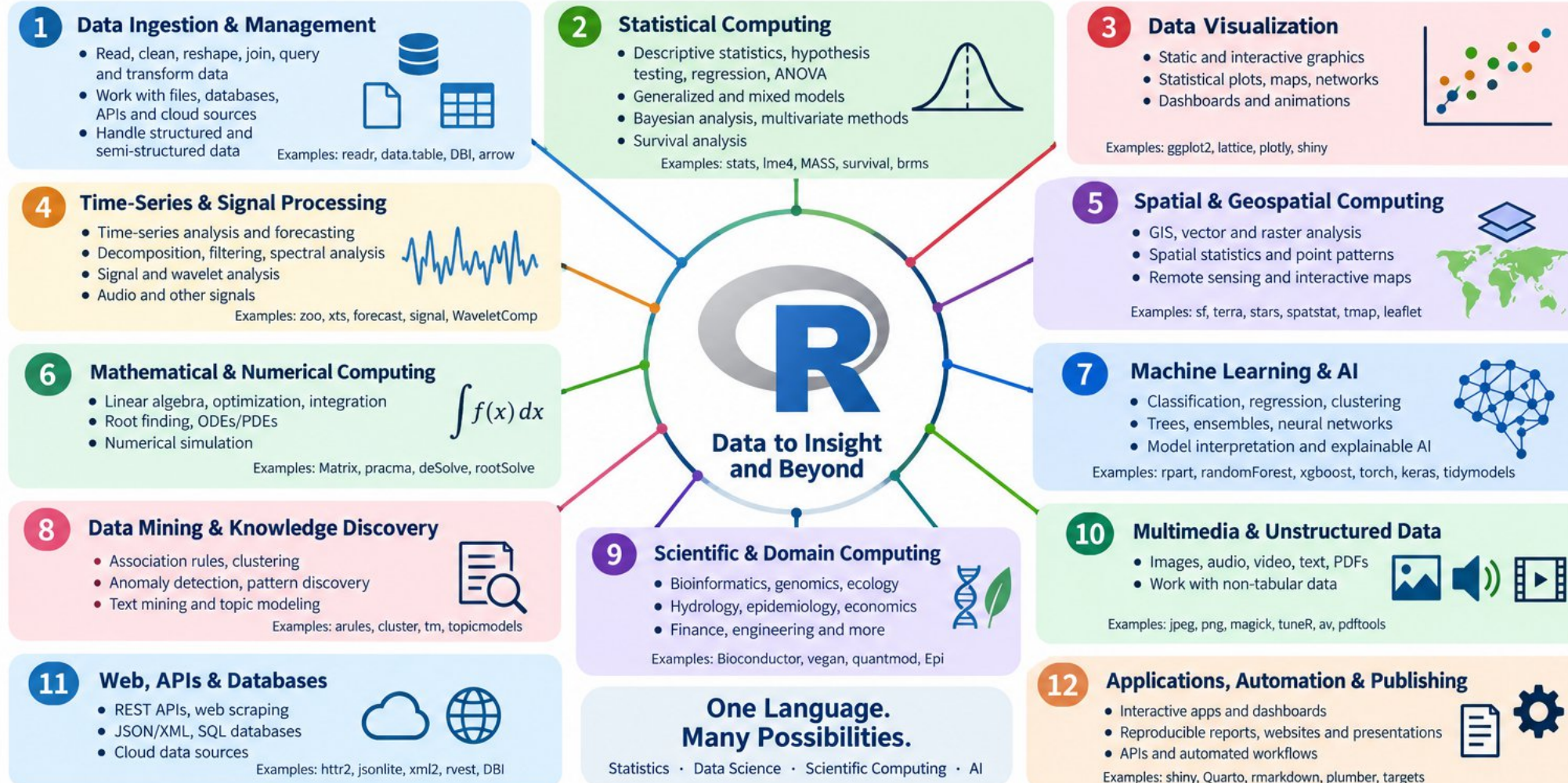
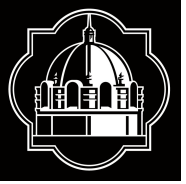
Genomics Analysis



Genomics data	What it represents	Analogous data structure	R / Bioconductor structure	Key library
Sequences	DNA, RNA, proteins	Text / character sequences	DNASTring, RNASTring, DNASTringSet	Biostrings
Genomic coordinates	Genes, exons, regulatory regions, variants	GIS vector features	GRanges	GenomicRanges
Expression / counts	Gene abundance across samples	Numeric matrix	matrix, assay matrix	SummarizedExperiment, DESeq2
Microarrays	Fluorescence/intensity measurements	Image / numeric array	Matrix/array + metadata	limma, oligo
Sequence reads	Millions of short DNA/RNA sequences + quality	Text records + metadata	ShortReadQ, DNASTringSet	ShortRead
Sequence alignments	Reads mapped onto genome	Coordinates + sequence data	BAM/SAM objects	Rsamtools, GenomicAlignments
Genetic variants	SNPs, insertions, deletions	GIS-like point/range features + attributes	VCF, VRanges	VariantAnnotation
Multiple assays	RNA, proteins, methylation, etc. for same samples	Multidimensional/relational data	MultiAssayExperiment	MultiAssayExperiment
Spatial genomics	Gene expression + tissue location	GIS + image + matrix	Spatial coordinates + assays/images	SpatialExperiment

What R Can Do

A versatile environment for data, statistics, modeling and more



R Works with Many
Kinds of Data



Tabular
Data



Time
Series



Spatial
Data



Text



Images



Audio



Video



Networks



Genomics

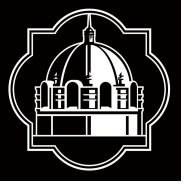


And More...

Acquire → Represent → Manipulate → Explore → Analyze → Model → Predict → Communicate → Automate

From data to discovery — with R.

Final Thoughts



- R offers a rich array of data structures
 - Build on a few basic data types
- Ability to read domain-specific data formats
- R Transforms data into few standard formats
 - Vector, Matrix, Array, List, Table
- Once transformed a rich set of tools become available for analysis

Domain	Sequence	Spatial	Matrix/Array	Metadata
Text	Words/tokens	—	Document-term matrix	Author/date
GIS	—	Coordinates/geometries	Raster	Feature attributes
Imaging	—	Pixel location	RGB/intensity array	Image properties
Audio	Time samples	—	Channels $\times$ samples	Sample rate
Video	Frame sequence	Pixel location	Frames $\times$ H $\times$ W $\times$ channels	FPS
Genomics	DNA/RNA	Genomic coordinates	Genes $\times$ samples	Gene/sample attributes

Notes, Data and Code can be found at: <https://github.com/vuddameri/thuRsday>

